

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE		N° réalisation : 2
Nom, prénom : BAUDELET Kaelian		N° candidat : 02249639424
Épreuve ponctuelle ☒	Contrôle en cours de formation ☐	Date : 11/04/2026
Organisation support de la réalisation professionnelle  Forum Forum créé par SimpleEduc		
Intitulé de la réalisation professionnelle Plateforme d'échange communautaire type forum, permettant aux étudiants de communiquer, de s'entraider et d'organiser des discussions par thématiques scolaires. L'écosystème est divisé en une partie backend Symfony API Platform et une application mobile Flutter.		
Période de réalisation : 08/10/25 au 12/12/25 Lieu : Dans le cadre de la formation à l'EPSI Modalité : ☒ Seul(e) ☐ En équipe		
Compétences travaillées ☒ Concevoir et développer une solution applicative ☒ Assurer la maintenance corrective ou évolutive d'une solution applicative ☒ Gérer les données		
Conditions de réalisation¹ (ressources fournies, résultats attendus) Ressources fournies : - Description du contexte - Description de l'existant - Expression des besoins - Code Existant - Script SQL, Schéma MCD/MLD de base données existant - Cahier des charges avec diagramme des classes proposé. Résultats attendus : - Application pleinement opérationnelle offrant une plateforme complète d'échange et d'entraide (création de forums et de sous-forums) avec un système de messages et de réponses pour les étudiants et les enseignants. Le projet comprend le développement d'une application mobile avec Flutter, la mise en place d'une API avec API Platform, la création de la structure de la base de données pour gérer la hiérarchie des messages et des forums, et la création d'un back-office. - Documentation utilisateur.		

¹ En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

Description des ressources documentaires, matérielles et logicielles utilisées²

Ressources documentaires :

- TPs réalisés en cours
- Cours Symfony
- Cours API Platform
- Cours Flutter
- Documentation : <https://symfony.com/doc/current/index.html>

Ressources matérielles :

- Ordinateur personnel
- Hébergement de l'application sur le site nuage-pedagogique.fr et <https://forumscolaire.kaelian.dev>
- Hébergement du code et de la documentation sur GitHub

Ressources logicielles :

- Framework : **Symfony 6.x + API Plateform**
- ORM : **Doctrine**
- SGBD/Base de données : **MariaDB/MySQL**
- Front-End: **Flutter**

Modalités d'accès aux productions³ et à leur documentation⁴

Accès à l'application Forum :

Page de l'application de démonstration : <https://forum.kaelian.dev>

Compte :

Identifiant : **test@demo.fr**

Mot de passe : **test**

Accès aux ressources documentaires du projet :

Page du projet : <https://portfolio-bts-sio.kaelian.dev/projects?project=22>

Accès à l'ensemble du projet :

Base de données :

Base de donnée (via DBGate) : <https://forumdb.kaelian.dev>

Identifiant : **forum**

Mot de passe : **sn3!JeKoqK3QX**

Code Source :

Github du projet : <https://github.com/kaelianbaudelet/forum>

² Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

³ Conformément au référentiel du BTS SIO « Dans tous les cas, les candidats doivent se munir des outils et ressources techniques nécessaires au déroulement de l'épreuve. Ils sont seuls responsables de la disponibilité et de la mise en œuvre de ces outils et ressources. La circulaire nationale d'organisation précise les conditions matérielles de déroulement des interrogations et les pénalités à appliquer aux candidats qui ne se seraient pas munis des éléments nécessaires au déroulement de l'épreuve. ». Les éléments peuvent être un identifiant, un mot de passe, une adresse réticulaire (URL) d'un espace de stockage et de la présentation de l'organisation du stockage.

⁴ Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation professionnelle, par exemples service fourni par la réalisation, interfaces utilisateurs, description des classes ou de la base de données.

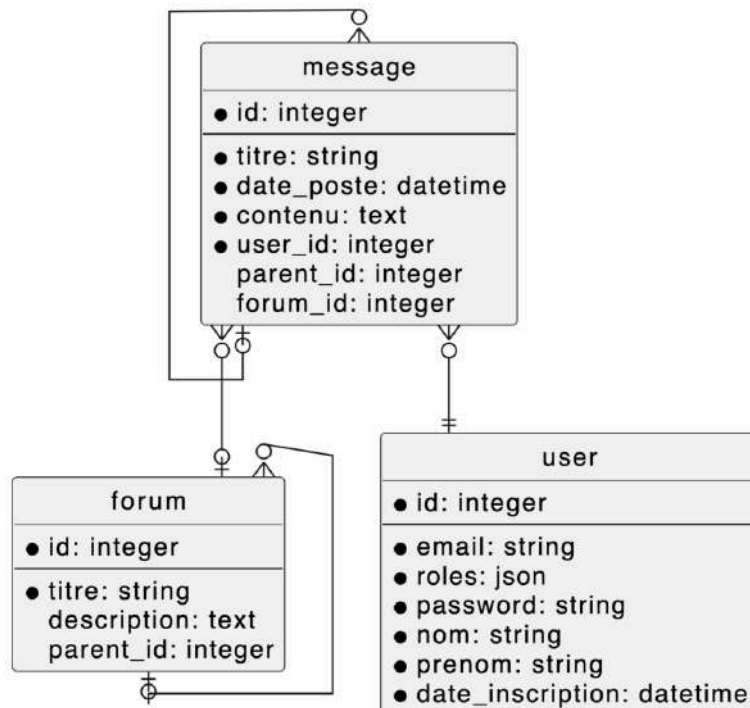
ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)

Épreuve E5 - Conception et développement d'applications (option SLAM)

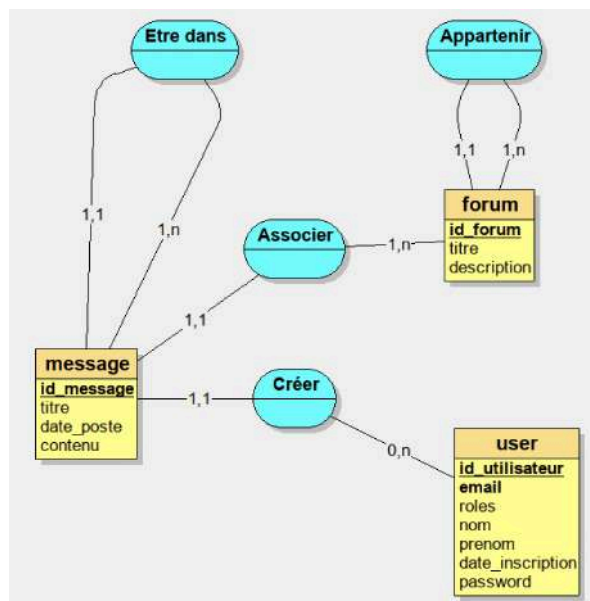
Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

MCD / MLD / Diagramme Entité-Relation :

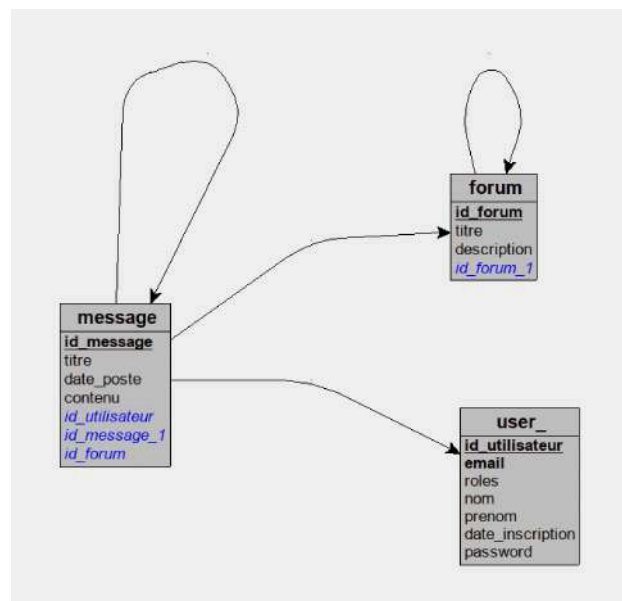
Diagramme Entité-Relation de la Base de donnée généré par l'ORM Doctrine



MCD de la Base de donnée rédigé et retraduit à la main



MLD de la Base de donnée rédigé et retraduit à la main



Fonctionnalités :

Note au jury : Bien que la documentation évoque un large éventail de fonctionnalités, ce rendu se concentre sur les fonctionnalités les plus essentielles et pertinentes.

1. Authentification via Token JWT

Le système permet une connexion sécurisée sans session côté serveur, idéale pour une application mobile. L'API délivre un jeton signé permettant d'identifier l'utilisateur sur Flutter à chaque requête.

Côté API Platform (config/packages/lexik_jwt_authentication.yaml):

```
lexik_jwt_authentication:
  secret_key: '%env(resolve:JWT_SECRET_KEY)%'
  public_key: '%env(resolve:JWT_PUBLIC_KEY)%'
  pass_phrase: '%env(JWT_PASSPHRASE)%'
  token_ttl: 3600
  token_extractors:
    authorization_header:
      enabled: true
      prefix: Bearer
      name: Authorization
    query_parameter:
      enabled: true
      name: bearer
```

2. Système de stockage sécurisé des secrets (Vault mobile)

Pour éviter le vol de session, les identifiants et tokens ne sont pas stockés dans les préférences classiques du téléphone mais dans une enclave sécurisée (Keystore/Keychain).

Côté Flutter (pubspec.yaml):

```
dependencies:
  flutter_secure_storage: ^9.0.0
```

Dans `loginUser` dans `auth_provider.dart` une fois que l'utilisateur authentifié, on garde en mémoire dans l'enclave sécurisée du téléphone le token JWT.

```
Future<void> loginUser(String email, String password) async {
  try {
    ...

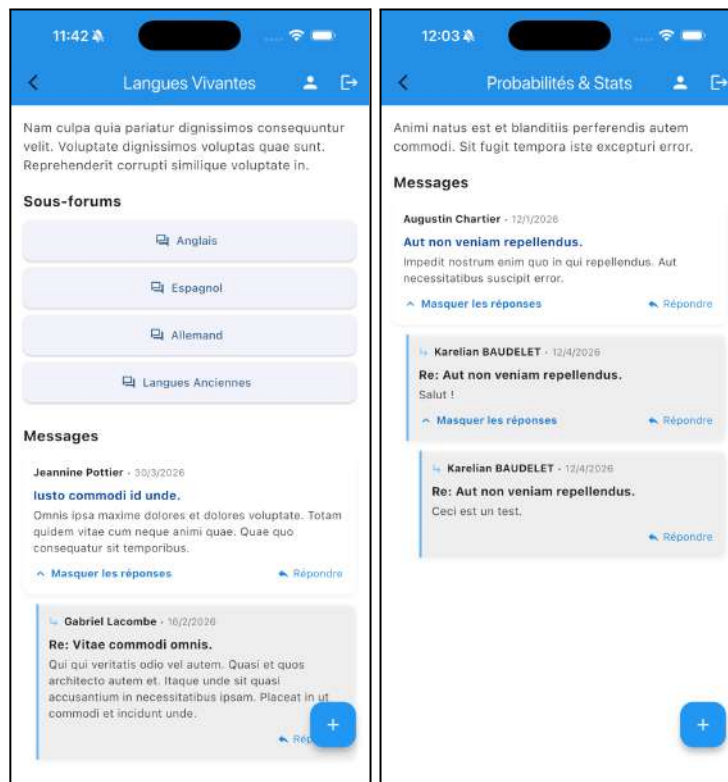
    final user = responseData['data'] ?? responseData['user'] ?? {};
    final firstName = user['prenom'] ?? '';
    final lastName = user['nom'] ?? '';

    await _secureStorage.saveToken(token);
    await _secureStorage.saveCredentials(email, password);
    await _secureStorage.saveUserInfo(firstName, lastName);

    _firstName = firstName;
    _lastName = lastName;
    _email = email;
    _isLoggedIn = true;
    notifyListeners();
  } catch (e) {
    rethrow;
  }
}
```

3. Système de réponses imbriquées (Commentaires de commentaires)

Mise en place d'une relation récursive en base de données permettant aux utilisateurs de répondre précisément à un message spécifique au sein d'un forum.



Côté **API Platform** (Entité Message.php) :

Dans l'entité `Message.php`, on utilise une relation auto-référencée. Cela permet de créer une structure hiérarchique où un message peut posséder un 'parent' et plusieurs 'enfants' (sous-message), permettant d'avoir plusieurs commentaires dans d'autres commentaires sans multiplier les tables en base de données

```
<?php
namespace App\Entity;
...
#[ApiResponse(paginationItemsPerPage: 10, operations: [
    new GetCollection(normalizationContext: ['groups' => 'message:list']),
    new Post(
        processor: \App\State\MessageStateProcessor::class,
        security: "is_granted('ROLE_USER')",
        normalizationContext: ['groups' => ['message:item']],
        denormalizationContext: ['groups' => ['message:write']]
    ),
    new Get(normalizationContext: ['groups' => 'message:item']),
    ...
]),)]
#[ApiFilter(SearchFilter::class, properties: ['user' => 'exact', 'forum' => 'exact', 'parent' => 'exact'])]
#[ApiFilter(ExistsFilter::class, properties: ['parent'])]
#[ApiFilter(OrderFilter::class, properties: ['id' => 'ASC', 'titre' => 'DESC'])]
#[ORM\Entity(repositoryClass: MessageRepository::class)]
class Message
```

```

{
    ...
    #[ORM\ManyToOne(targetEntity: self::class, inversedBy: 'replies')]
    #[ORM\JoinColumn(onDelete: 'SET NULL')]
    #[Groups(['message:list', 'message:item', 'message:write'])]
    private ?self $parent = null;
    /**
     * @var Collection<int, self>
     */
    #[ORM\OneToMany(targetEntity: self::class, mappedBy: 'parent')]
    #[Groups(['message:item', 'message:list'])]
    private Collection $replies;
    #[ORM\ManyToOne(targetEntity: Forum::class, inversedBy: 'messages')]
    #[ORM\JoinColumn(nullable: true)]
    #[Groups(['message:list', 'message:item', 'message:write'])]
    private ?Forum $forum = null;
    ...
    public function getParent(): ?self
    {
        return $this->parent;
    }
    public function setParent(?self $parent): static
    {
        $this->parent = $parent;
        return $this;
    }
    ...
    /**
     * @return Collection<int, self>
     */
    public function getReplies(): Collection
    {
        return $this->replies;
    }
    public function addReply(self $reply): static
    {
        if (!$this->replies->contains($reply)) {
            $this->replies->add($reply);
            $reply->setParent($this);
        }
        return $this;
    }
    ...
}

```

Côté Flutter (Widget `_buildMessageItem` dans `forum_detail_screen.dart`):

Dans l'écran `forum_detail_screen.dart`, dans le widget `_buildMessageItem` on utilise une méthode récursive lors de la construction de l'interface d'un message et l'affichage de ses enfants.

```

Widget _buildMessageItem(MessageModel m, int forumId, {double indent = 0}) {
    final bool isExpanded = _expandedIds.contains(m.id);
    final List<MessageModel>? cachedReplies = _loadedReplies[m.id];

    return Column(

```

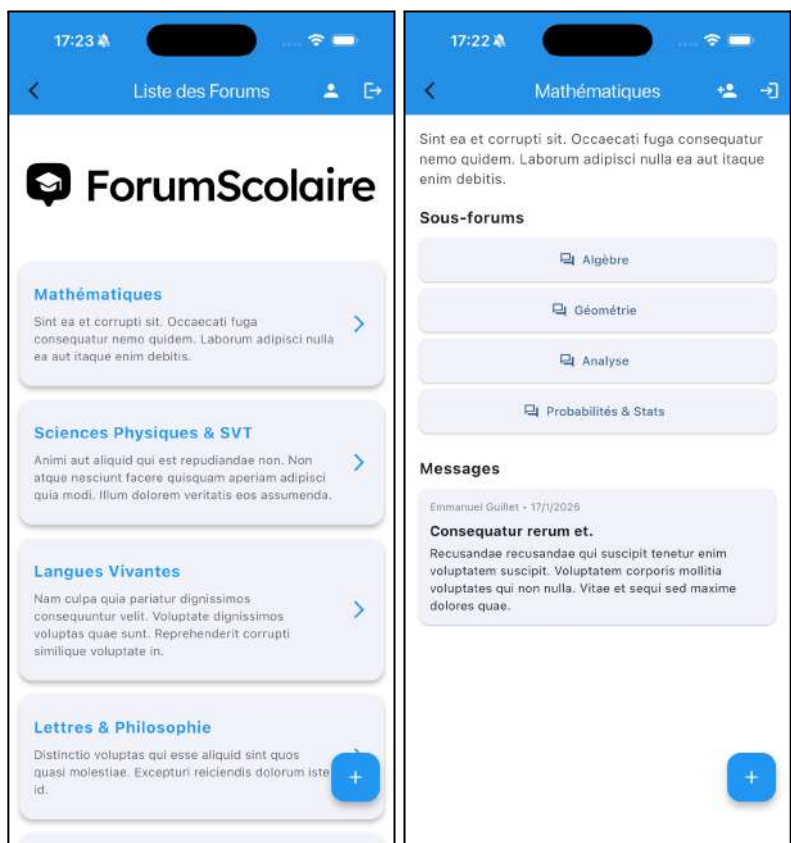
```

children: [
  Padding(
    padding: EdgeInsets.only(left: indent),
    child: _MessageCard(
      message: m,
      indent: indent,
      isExpanded: isExpanded,
      forumId: forumId,
      onToggleExpand: () async {
        if (isExpanded) {
          setState(() => _expandedIds.remove(m.id));
        } else {
          // If not loaded, fetch from API
          if (cachedReplies == null) {
            setState(() => _expandedIds.add(m.id));
            try {
              final replies = await MessageApi().fetchReplies(m.id);
              setState(() {
                _loadedReplies[m.id] = replies;
              });
            } catch (e) {
              if (mounted) {
                ScaffoldMessenger.of(context).showSnackBar(
                  SnackBar(content: Text('Erreur chargement réponses : $e')),
                );
              }
            }
          } else {
            setState(() => _expandedIds.add(m.id));
          }
        }
      },
      onReplySuccess: () async {
        if (m.parentId == null) {
          // Si on a répondu à un message racine, on recharge la liste des racines
          setState(() {
            futureMessages = MessageApi().fetchRootMessagesByForum(forumId);
          });
        } else {
          // Si on a répondu à une réponse, on recharge la liste des réponses du parent de 'm'
          // pour que 'm' soit actualisé avec son nouveau compteur de réponses
          try {
            final parentReplies = await MessageApi().fetchReplies(m.parentId!);
            setState(() {
              _loadedReplies[m.parentId!] = parentReplies;
            });
          } catch (e) {
            // Fallback: refresh root if something fails
            setState(() {
              futureMessages = MessageApi().fetchRootMessagesByForum(forumId);
            });
          }
        }
      },
    ),
  ),
  if (isExpanded)
    cachedReplies == null
      ? const Center(child: Padding(padding: EdgeInsets.all(8.0), child:
CircularProgressIndicator(strokeWidth: 2)))
      : Column(
        children: cachedReplies.map((reply) => _buildMessageItem(reply, forumId, indent:
indent + 16)).toList(),
      ),
    ],
  );
}
}

```

4. Structure de Forum récursive (Forum dans Forum)

Le système permet une organisation hiérarchique sans limite de profondeur. Un forum peut contenir d'autres forums, permettant une spécialisation précise des espaces de discussion.



Côté **API Platform** (Entité Forum.php) :

Dans l'entité `Forum.php`, on utilise une relation auto-référencée. Cela permet de créer une structure hiérarchique où un forum peut posséder un 'parent' et plusieurs 'enfants' (sous-forums), permettant une organisation flexible des catégories sans multiplier les tables en base de données

```
<?php

namespace App\Entity;

...

#[ApiResource(
    normalizationContext: ['groups' => ['forum:read']],
    denormalizationContext: ['groups' => ['forum:write']],
    paginationItemsPerPage: 100,
    operations: [
        new GetCollection(normalizationContext: ['groups' => ['forum:list']]),
        new Get(normalizationContext: ['groups' => ['forum:read']]),
        ...
    ],
)]

#[ORM\Entity(repositoryClass: ForumRepository::class)]
class Forum
{
    ...

    #[ORM\ManyToOne(targetEntity: self::class, inversedBy: 'children')]
```

```

#[ORM\JoinColumn(name: 'parent_id', referencedColumnName: 'id', onDelete: 'CASCADE')]
#[Groups(['forum:read', 'forum:list', 'forum:write'])]
private ?self $parent = null;
/**
 * @var Collection<int, self>
 */
#[ORM\OneToMany(targetEntity: self::class, mappedBy: 'parent')]
#[Groups(['forum:read', 'forum:list'])]
private Collection $children;
/**
 * @var Collection<int, Message>
 */
#[ORM\OneToMany(targetEntity: Message::class, mappedBy: 'forum')]
#[Groups(['forum:read'])]
private Collection $messages;
...
public function getParent(): ?self
{
    return $this->parent;
}

public function setParent(?self $parent): static
{
    $this->parent = $parent;

    return $this;
}
/**
 * @return Collection<int, self>
 */
public function getChildren(): Collection
{
    return $this->children;
}
public function addChild(self $child): static
{
    if (!$this->children->contains($child)) {
        if ($this->children->count() >= 1) {
            throw new \RuntimeException('Un forum ne peut avoir qu\'un seul sous-forum.');
        }
        $this->children->add($child);
        $child->setParent($this);
    }

    return $this;
}
...

```

Côté Flutter (Widget `_buildMessageItem` dans `forum_detail_screen.dart`):

Dans l'écran `forum_detail_screen.dart`, dans le widget `build` on réutilise le même écran (`//forum-detail`) en lui passant simplement l'ID du niveau suivant.

```
Widget build(BuildContext context) {
```

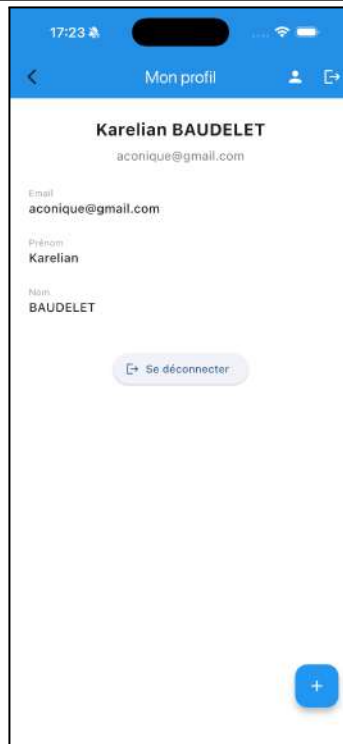
```

return FutureBuilder<ForumModel>(
  future: futureForum,
  builder: (context, forumSnapshot) {
    ...
    final forum = forumSnapshot.data!;
    return MyScaffold(
      ...
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            ...
            // Section Sous-forums
            if (forum.children.isNotEmpty) ...[
              const Text(
                'Sous-forums',
                style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 10),
              ...forum.children.map((child) => Padding(
                padding: const EdgeInsets.only(bottom: 8.0),
                child: ElevatedButton.icon(
                  onPressed: () {
                    Navigator.pushNamed(
                      context,
                      '/forum-detail',
                      arguments: child.id,
                    );
                  },
                  icon: const Icon(Icons.forum_outlined),
                  label: Text(child.titre),
                  style: ElevatedButton.styleFrom(
                    minimumSize: const Size(double.infinity, 50),
                    shape: RoundedRectangleBorder(
                      borderRadius: BorderRadius.circular(10),
                    ),
                  ),
                ),
              ),
            ],
            const SizedBox(height: 20),
          ],
        ),
      ),
    ),
  ),
);
}

```

5. Espace profil utilisateur

Chaque membre dispose d'une page dédiée affichant ses informations d'identité (Nom, Prénom, Email). Les données sont récupérées dynamiquement depuis l'API via le token d'authentification.



Côté Flutter (auth_provider.dart):

Dans le provider `auth_provider.dart` au moment d'authentifier un utilisateur on stock les information de profil dans `_firstName`, `_lastName`, `_email`.

```
Future<void> loginUser(String email, String password) async {
  try {
    final response = await login(email, password);
    final responseData = jsonDecode(response.body);
    final token = responseData['token'] ?? '';
    final user = responseData['data'] ?? responseData['user'] ?? {};
    final firstName = user['prenom'] ?? '';
    final lastName = user['nom'] ?? '';
    await _secureStorage.saveToken(token);
    await _secureStorage.saveCredentials(email, password);
    await _secureStorage.saveUserInfo(firstName, lastName);
    _firstName = firstName;
    _lastName = lastName;
    _email = email;
    _isLoggedIn = true;
    notifyListeners();
  } catch (e) {
    rethrow;
  }
}
```

Dans l'écran `auth_provider.dart` on récupère les information de profil utilisateur stockée par le `AuthProvider` pour les afficher.

...

```
class ProfileScreen extends StatelessWidget {
  const ProfileScreen({super.key});
  @override
  Widget build(BuildContext context) {
    // We can use AuthProvider which already has the info loaded from SecureStorage
    final authProvider = Provider.of<AuthProvider>(context);
    return MyScaffold(
      name: 'Mon profil',
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(24.0),
        child: Column(
          children: [
            ...
            const SizedBox(height: 16),
            _buildInfoRow('Email', authProvider.email ?? ''),
          ],
        ),
      ),
    );
  }
}
```

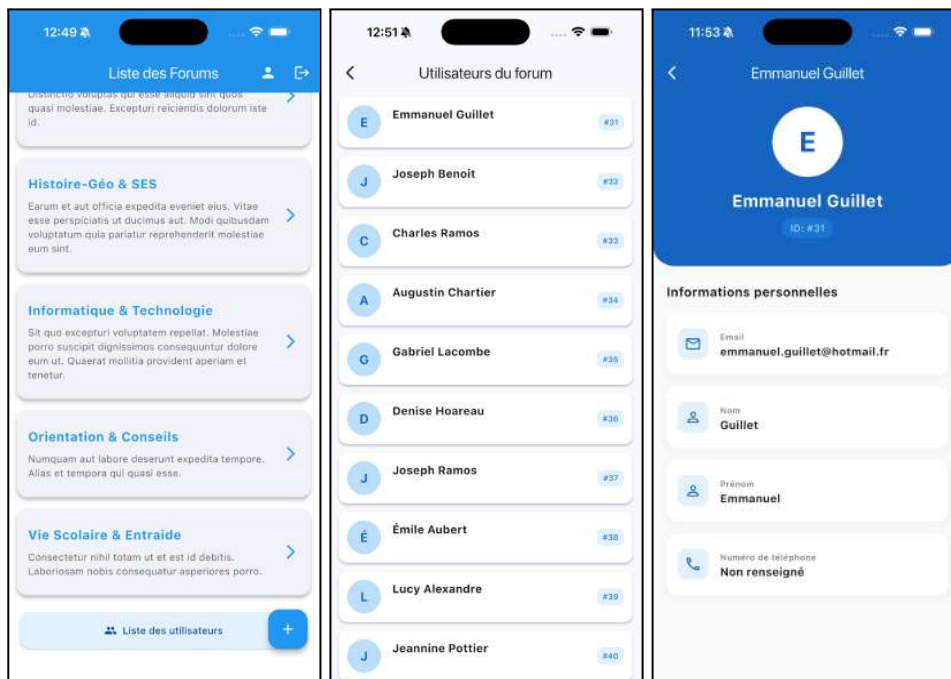
```

        _buildInfoRow('Prénom', authProvider.firstName ?? ''),
        _buildInfoRow('Nom', authProvider.lastName ?? ''),
        ...
    ],
),
),
);
}
...
}

```

5. Liste des utilisateurs de ForumScolaire

Chaque membre dispose d'une page dédiée affichant ses informations d'identité (Nom, Prénom, Email). Les données sont récupérées dynamiquement depuis l'API via le token d'authentification.



Côté **API Platform** (Entité User.php) :

Au sein de l'entité User.php, nous avons mis en place une ressource API pour la gestion des membres. Grâce à l'utilisation des opérations de collection et du groupe de sérialisation user:list, l'API expose une liste de membres sécurisée ne diffusant que les informations nécessaires.

```

<?php
namespace App\Entity;
...
#[ApiResource(operations: [
    new GetCollection(normalizationContext: ['groups' => 'user:list']),
    new Post(processor: UserStateProcessor::class, denormalizationContext: ['groups' =>
['user:write']]),
    new Get(normalizationContext: ['groups' => 'user:item']),
    new Put(),
    new Patch(security: "is_granted('ROLE_ADMIN') or object == user"),
    new Delete(),
],)]
#[ApiFilter(SearchFilter::class, properties: ['id' => 'exact', 'nom' => 'exact', 'prenom' =>
'partial'])]
#[ApiFilter(OrderFilter::class, properties: ['id' => 'ASC', 'titre' => 'DESC'])]
#[ORM\Entity(repositoryClass: UserRepository::class)]
#[ORM\UniqueConstraint(name: 'UNIQ_IDENTIFIER_EMAIL', fields: ['email'])]
class User implements UserInterface, PasswordAuthenticatedUserInterface
{
    #[ORM\Id]

```

```

#[ORM\GeneratedValue]
#[ORM\Column]
#[Groups(['user:list', 'user:item'])]
private ?int $id = null;
#[ORM\Column(length: 180)]
#[Groups(['user:item', 'user:write'])]
#[Assert\NotBlank(message: "L'email est obligatoire.")]
#[Assert\Email(message: "L'email n'est pas valide.")]
private ?string $email = null;
/**
 * @var list<string> The user roles
 */
#[ORM\Column]
private array $roles = [];
/**
 * @var string The hashed password
 */
#[ORM\Column]
#[Groups(['user:write'])]
#[Assert\NotBlank(message: "Le mot de passe est obligatoire.")]
#[Assert\Regex(
    pattern: "/^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[A-Za-z0-9]).{12,}$/",
    message: "Le mot de passe doit contenir au moins 12 caractères, une majuscule, une minuscule,
un chiffre et un caractère spécial."
)]
private ?string $password = null;

#[ORM\Column(length: 30)]
#[Groups(['user:list', 'user:item', 'message:list', 'message:item', 'user:write', 'forum:read'])]
#[Assert\NotBlank(message: "Le nom est obligatoire.")]
#[Assert\Regex(
    pattern: "/^[a-zA-ZÀ-ÿ '-]+$/u",
    message: "Le nom ne doit contenir que des lettres."
)]
private ?string $nom = null;
#[ORM\Column(length: 30)]
#[Groups(['user:list', 'user:item', 'message:list', 'message:item', 'user:write', 'forum:read'])]
#[Assert\NotBlank(message: "Le prénom est obligatoire.")]
#[Assert\Regex(
    pattern: "/^[a-zA-ZÀ-ÿ '-]+$/u",
    message: "Le prénom ne doit contenir que des lettres."
)]
private ?string $prenom = null;

#[ORM\Column]
private ?\DateTime $dateInscription = null;
/**
 * @var Collection<int, Message>
 */
#[ORM\OneToMany(targetEntity: Message::class, mappedBy: 'user', orphanRemoval: true)]
private Collection $messages;
...
}

```

Côté Flutter (users_screen.dart et user_list_api.dart):

Dans le script de récupération des utilisateurs user_list_api.dart on fait appel à l'api Symfony pour récupérer les utilisateurs.

```

...
class UserListApi {
    final String baseUrl = dotenv.env['API_BASE_URL']!;
    // Méthode asynchrone qui renvoie une liste d'objets UserModel
    Future<List<UserModel>> fetchUsers() async {
        // Construit l'URL complète vers l'endpoint (la route) /users
        final url = "$baseUrl/users";
        // Envoie une requête HTTP GET vers l'API et attend la réponse
        final response = await http.get(Uri.parse(url));

        // Vérifie si la réponse HTTP est correcte (code 200)
        if (response.statusCode == 200) {
            // Convertit le JSON brut (string) en Map (objet clé/valeur)
            final Map<String, dynamic> data = json.decode(response.body);

            // Récupère le tableau d'utilisateurs dans la clé "member"
            // Si "member" n'existe pas, retourne une liste vide
            final List<dynamic> members = data['member'] ?? [];

            // Transforme chaque élément JSON en un objet UserModel
            // puis renvoie la liste complète

```

```

        return members.map((item) => UserModel.fromJson(item)).toList();
    } else {
        throw Exception('Erreur HTTP ${response.statusCode}');
    }
}
}
}

```

Dans l'écran `users_screen.dart` on affiche les utilisateurs inscrits sur la plateforme.

...

```

class UsersScreen extends StatefulWidget {
  const UsersScreen({super.key});
  @override
  State<UsersScreen> createState() => _UsersScreenState();
}

class _UsersScreenState extends State<UsersScreen> {
  // Future contenant la liste des utilisateurs (chargés depuis l'API)
  late Future<List<UserModel>> futureUsers;
  @override
  void initState() {
    super.initState();
    // Appel de l'API dès l'ouverture de l'écran
    futureUsers = UserListApi().fetchUsers();
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // Barre supérieure de la page
      appBar: AppBar(title: const Text('Utilisateurs du forum')),
      // FutureBuilder permet d'afficher du contenu une fois la Future terminée
      body: FutureBuilder<List<UserModel>>(
        future: futureUsers,
        // builder = fonction appelée à chaque changement d'état du Future
        builder: (context, snapshot) {
          ...
          // Liste des utilisateurs une fois chargés
          final users = snapshot.data!;
          // Affichage de la liste sous forme de ListView
          return ListView.builder(
            itemCount: users.length, // nombre d'utilisateurs
            itemBuilder: (context, index) {
              final u = users[index]; // utilisateur courant
              return Card(
                margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
                color: Colors.white,
                elevation: 2,
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(12),
                ),

                child: Padding(
                  padding: const EdgeInsets.all(12),
                  child: Row(
                    children: [
                      // Avatar avec l'initiale du prénom
                      ...
                      // Informations de l'utilisateur
                      Expanded(
                        child: Column(
                          crossAxisAlignment: CrossAxisAlignment.start,
                          children: [
                            Text(
                              "${u.firstName} ${u.lastName}",
                              style: const TextStyle(
                                fontSize: 16,
                                fontWeight: FontWeight.bold,
                              ),
                            ),
                            const SizedBox(height: 4),
                            Text(
                              u.email,
                              style: TextStyle(
                                fontSize: 13,
                                color: Colors.grey[600],
                              ),
                            ),
                          ],
                        ),
                      1,
                    ],
                  ),
                ),
                // Badge avec l'ID
                ...

```

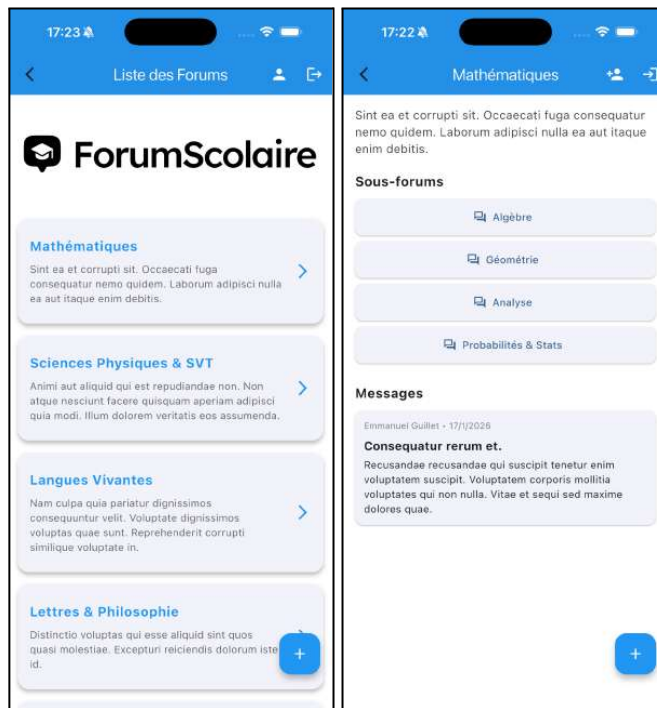
Users Stories :

US01 - En tant qu'utilisateur, je veux naviguer à travers les forums et sous-forums afin de trouver des discussions spécifiques à une matière ou un sujet.

- **Critères d'acceptation :**

- Affichage de la liste des forums principaux sur l'accueil.
- Possibilité de cliquer sur un forum pour voir ses sous-forums.

Attendu :

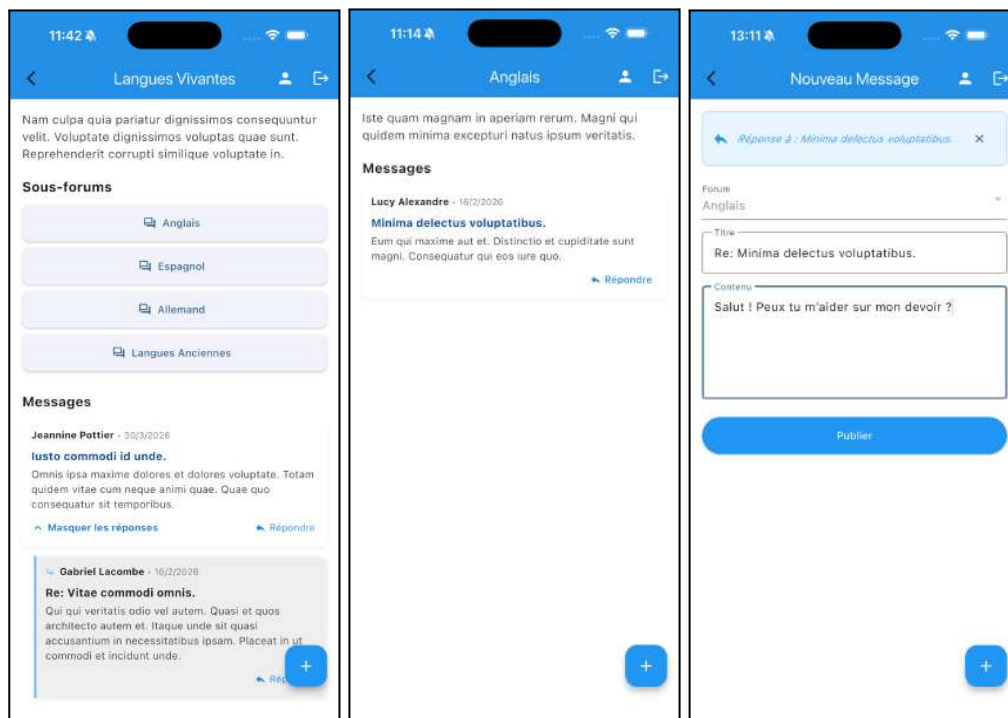


US02 - En tant qu'utilisateur, je veux pouvoir répondre au message d'un autre étudiant ou enseignant pour l'aider.

Critères d'acceptation :

- Bouton "Répondre" sur chaque message.
- Le message de réponse doit être rattaché au "Parent ID" du message d'origine (gestion de l'arborescence).
- Affichage indenté ou chronologique des réponses.

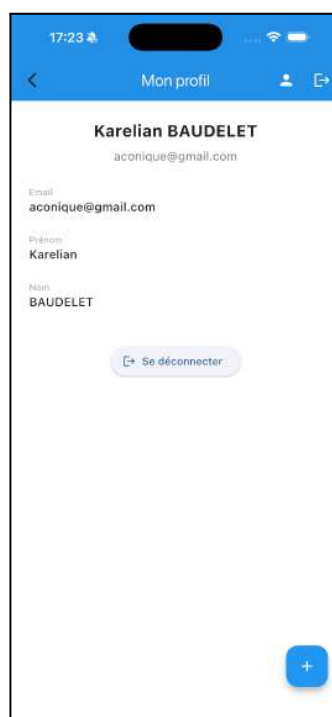
Attendu :



US03 - En tant qu'utilisateur, je veux accéder à mon profil pour voir mes informations personnelles.

- **Critères d'acceptation :**
 - Affichage du nom, prénom et email.
 - Option de déconnexion.

Attendu :



US04 - En tant qu'utilisateur, je veux voir la liste des membres de la plateforme afin de connaître les participants.

- Critères d'acceptation :

- Affichage d'un annuaire des utilisateurs (Nom, Prénom, Email, Numéro de Téléphone).

Attendu :

